

Arrays

Page No. 103
Date: / /

1. What is an Array
2. Declaring and initializing
3. Accessing Array

1. What is an Array

Scalar $\rightarrow$ int $x = 10;$

10

100/101

Vector $\rightarrow$ int $A[5];$

0	1	2	3	4

$A[2] = 15;$

20/1, 20/5, 4/5, 6/7, 8/9

2. Declaration & initialization

① int $A[5];$

0	1	2	3	4
?	?	?	?	?

garbage

② int $A[5] = \{2, 4, 6, 8, 10\};$

2	4	6	8	10
---	---	---	---	----

③ int $A[5] = \{2, 2, 4\};$

0	1	2	3	4
2	2	4	0	0

④ int $A[5] = \{20\};$

0	1	2	3	4
0	0	0	0	0

⑤ int $A[] = \{2, 4, 6, 8, 10\};$

0	1	2	3	4	5
2	4	6	8	10	

3. Accessing

int A[5] = {2, 5, 4, 9, 8};

for (i=0; i<5; i++)

{ printf ("%d", A[i]);

	0	1	2	3	4
A	2	5	4	9	8
addr	300	308	316	324	332

printf ("%d", A[2]);
printf ("%d", 2[A]);
printf ("%d", *(A+2));

Static vs Dynamic Array → size of array is not constant

↓
size of array is constant

void main()

{
int A[5]; → in C, size has to be decided during compile time only

int n;
cin >> n;
int B[n];

→ size of array in C++ can be decided at runtime

C++ → P = new int[5];

C → P = (int*) malloc(5 * sizeof(int));

C++ → delete [] P;

C → free(P);

How to increase array size

int * p = new int[5];

int * q = new int[10];

** Copying elements of P to q

for (i=0; i<5; i++)

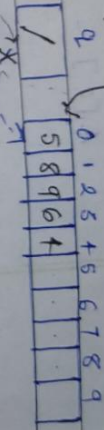
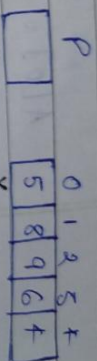
q[i] = p[i]

delete[] p;

**

p = q;

q = NULL



2D Array

Methods:-

① int A[A][4] = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12}, {13,14,15,16} };

4x4

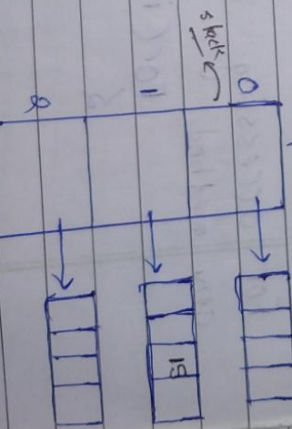
stack

② int * A[5];

array of pointers

heap

A	0	1	2	3
0	200/1	2/3	4/9	6/7
1	8/9	10/11	15	12/13
2				
3				



initializⁿ:- A[1][2] = 15;

A[0] = new int[4];

A[1] = new int[4];

A[2] = new int[4];

→ Both rows column in stack

initializⁿ method A[1][2] = 15;

is same as ①

→ rows in stack, column in heap

③

int ** A;

Array of pointers

A = new int*[5];

A = (int**) malloc(5 * Size of (int))

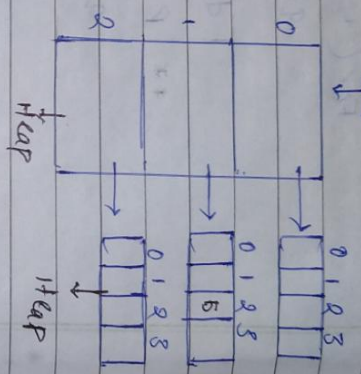
A[0] = new int[4];

int array

A[1] = new int[4];

A[2] = new int[4];

initialising A[1][2] = 15;
is same as 1st



* This method is followed for creating both thing
inside Heap only

For accessing all element use ^{double} For loop created by
any method

for(i=0; i<5; i++)

{ for(j=0; j<4; j++)

{ A[i][j] = 2

}

}

Array in Compiler

int A[5] = {3, 5, 8, 4, 2};

A	0	1	2	3	4
	3	5	8	4	2
	200/1	2/3	204/5	6/7	208/9

base address A[3] = 10;

$$\text{Add}(A[3]) = 200 + 3 * 2 = 206$$

$$\text{Add}(A[3]) = L_0 + 3 * 2$$

Addresses can't be known at compile time, so compiler writes (uses) formula for obtaining address

Addresses can't be known at compile time, so compiler writes (uses) formula for obtaining address

int A[1..5] $\rightarrow$ Some language indices start from 1 (used to be 0 earlier language).

$$\text{Add}(A[3]) = 200 + (3-1) * 2 = 204$$

$$\text{Add}(A[i]) = L_0 + (i-1) * \text{size of data}$$

it has 3 operⁿ

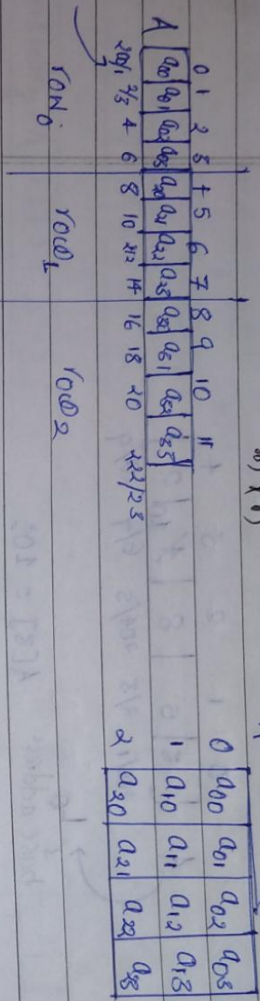
$L_0 + i * N$

it has 2 operⁿ

Due to less no. of operation C/C++ allows starting of indices from 0 only

Ram Major Formula For 2D arrays

int A[5][4];



$$\text{Add}(A[i][j]) = 200 + [i * 4 + j] * 2$$

$$= 200 + 6 * 2 = 212$$

$$\text{Add}(A[2][3]) = 200 + [2 * 4 + 3] * 2 = 222$$

$$A[i][j] = l_0 + [i * n + j] * w$$

IF language allowing indices from 1

int A[1...5][1...4]

$$A[i][j] = l_0 + [(i-1) * n + (j-1)] * w$$

having the extra operation

Column Major Formula for 2D Array

	0	1	2	3	4	5	6	7	8	9	10	11
Row	a ₀₀	a ₀₁	a ₀₂	a ₀₃	a ₀₄	a ₀₅	a ₀₆	a ₀₇	a ₀₈	a ₀₉	a ₀₁₀	a ₀₁₁
Row	a ₁₀	a ₁₁	a ₁₂	a ₁₃	a ₁₄	a ₁₅	a ₁₆	a ₁₇	a ₁₈	a ₁₉	a ₁₁₀	a ₁₁₁
Row	a ₂₀	a ₂₁	a ₂₂	a ₂₃	a ₂₄	a ₂₅	a ₂₆	a ₂₇	a ₂₈	a ₂₉	a ₂₁₀	a ₂₁₁
Col	0	1	2	3	4	5	6	7	8	9	10	11

$$\text{Addr}(A[i][j]) = 200 + [2*5+1]*2 = 214$$

$$\text{Addr}(A[i][3]) = 200 + [3*5+1]*2 = 220$$

Column
Row major $\rightarrow \text{Addr}(A[i][j]) = L_0 + [i*n + j] * w$

Row
Column major $\rightarrow \text{Addr}(A[i][j]) = L_0 + [i*n + j] * w$

While designing a compiler it is your choice whether you follow Row major formula or column major formula as both having same no. of operations

Formula for 2D Arrays

Type $A[d_1][d_2][d_3][d_4]$

Row-major

$$\text{May to Row } A[i_1][i_2][i_3][i_4] = L_0 + [i_1*d_2*d_3*d_4 + i_2*d_3*d_4 + i_3*d_4 + i_4]*w$$

For Row-major move right to left
For Column-major move left to right

Column-major

$$A[i_1][i_2][i_3][i_4] = L_0 + [i_1*d_2*d_3*d_4 + i_2*d_3*d_4 + i_3*d_4 + i_4]*w$$

$$A[i_1][i_2][i_3][i_4] = l_0 + [i_1 * d_2 * d_3 * d_4 + i_2 * d_3 * d_4 + i_3 * d_4 + i_4] * m$$

no of multiplication $\rightarrow$ 3 2 1

$$4D \rightarrow 3 + 2 + 1$$

$$5D \rightarrow 4 + 3 + 2 + 1$$

$$nD \rightarrow n + (n-1) + \dots + 8 + 2 + 1 = \frac{n(n+1)}{2}$$

$$n(n+1) = \frac{n(n+1)}{2} = O(n^2)$$

is there any way to reduce the number of multiplication

Knuth's Rule

$$A[i_1][i_2][i_3][i_4] = l_0 + [i_1 * d_2 * d_3 * d_4 + i_2 * d_3 * d_4 + i_3 * d_4 + i_4] * m$$

Knuth's Rule

$$i_4 + i_3 * d_4 + i_2 * d_3 * d_4 + i_1 * d_2 * d_3 * d_4$$

$$= i_4 + d_4 * [i_3 + i_2 * d_3 + i_1 * d_2 * d_3]$$

$$= i_4 + d_4 * [i_3 + d_3 * [i_2 + i_1 * d_2]]$$

only 3 multiplication

$$4D \rightarrow 3$$

$$5D \rightarrow 4$$

$$nD \rightarrow n-1 \rightarrow O(n)$$

Work
 $A[5][2] = \{3420, 206, 330, 106, 50, 606, 370, 806, 390, 1006\}$
 $\rightarrow A[0]$ or $\ast(A+0)$ gives the base address of 1-D Array
 $\rightarrow B[i] \Rightarrow \ast(B+i) \rightarrow$ 1-D Array, gives base address of i th element
 $\rightarrow A[4][1] \Rightarrow \ast(\ast(A+4) + 1)$

Formula For 3D Arrays

Row-major

$$\text{Addr}(\downarrow A[i][j][k]) = L_0 + [i \ast m \ast n + j \ast n + k] \ast w$$

Column-major

$$\text{Addr}(A[i][j][k]) = L_0 + [k \ast l \ast m + j \ast l + i] \ast w$$

Note: C++ Compiler Follow Row-major matrix

Q. What is the output of following C code?

Assume that the address of x is 2000 (in decimal) and an integer requires four bytes of memory.

int main()

{
 unsigned int $x[A][3] = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$

printf("r.c, r.c, r.c", $x+3$, $\ast(x+5)$, $\ast(x+2)+5$)

}

For (Accessing) $A[2][1]$

2000

0	1	2	3
0	1	2	3
4	5	6	7
8	9	10	11

$$\ast(\ast(x+2) + 1)$$

$$\ast(x+2) = x+2$$

$$\ast(\ast(x+2) + 3) \rightarrow 10$$

$$\ast(x+2)+3 \rightarrow 2036$$

$$\ast(x+5) \rightarrow 2036$$

$$x+3 \rightarrow 2036$$

$\text{Cof} [1 \ 1 \ 1 \ 1 + 0 \ 1 \ 1] + 0 \cdot 1 = (1 \cdot 2)(1 \cdot 1) = 2$

$\text{Cof} [1 \ 1 \ 1 \ 1 + 0 \ 1 \ 1] + 0 \cdot 1 = (1 \cdot 2)(1 \cdot 1) = 2$

The given matrix is a square matrix of order 4. The elements are 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1. The matrix is symmetric.

$(1 \ 1 \ 1 \ 1) \times (1 \ 1 \ 1 \ 1) = (1 \ 1 \ 1 \ 1) \times (1 \ 1 \ 1 \ 1)$

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

$(1 + (1 \cdot 1) \cdot 1) \cdot 1$

$0 \leftarrow (1 + (1 \cdot 1) \cdot 1) \cdot 1$
 $0 \leftarrow (1 + (1 \cdot 1) \cdot 1) \cdot 1$
 $0 \leftarrow (1 + (1 \cdot 1) \cdot 1) \cdot 1$

```
# bool sortBySec (pair<int,int> a, pair<int,int> b)
{
    if (a.second != b.second)
        return (a.second > b.second)
    else
        return (a.first < b.first);
}

vector<pair<int,int>> v;
sort(v.begin(), v.end(), sortBySec);
```

non-overlapping sub array $\rightarrow$ contiguous sub-array

$(b[0] < a[0] < b[1] < a[1] < b[2] < a[2])$ and
 $(a[0] < b[0] < a[1] < b[1] < a[2] < b[2])$

→ #include <stdio.h>

#include <stdlib.h>

struct Array

{ int *A;

int size;

int length;

};

void Display (struct Array arr)

{

int i;

printf (" % Elements are \n");

for (i = 0; i < arr.length; i++)

printf ("%d", arr.A[i]);

}

int main()

{

struct Array arr;

int n, i;

printf ("Enter size of an array");

scanf ("%d", &arr.size);

arr.A = (int *) malloc (arr.size * sizeof(int));

arr.length = 0;

printf ("Enter all elements \n");

```

For (i=0; i<n; i++)
    scanf("%d", &arr[i]);

```

```

arr.length = n;

```

```

display(arr);

```

```

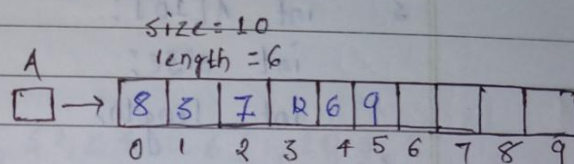
return 0;

```

```

{

```



1. Display

```

For (i=0; i<length; i++)

```

```

{

```

```

    Print (A[i]);

```

```

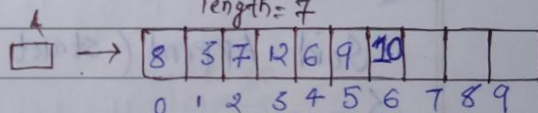
}

```

size = 10

length = 7

2. Add(x) / Append(x)



```

A[Length] = x;

```

```

Length++;

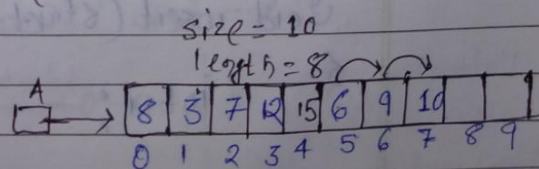
```

$f(n) = 2$

$f(n) = 2n^0$

$O(n^0) = O(1)$

3. Insert (4, 15)



Min² Max²
0 - n

```

For (i=length; i>index; i--)

```

```

    A[i] = A[i-1];

```

```

    A[index] = x;

```

```

    Length++;

```

$O(1) \rightarrow \text{Min}^2$
 $O(n) \rightarrow \text{Max}^2$

index should not go beyond size & length. check this by apply IF-else

Ex:-

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Array
```

```
{
    int A[10];
    int size;
    int length;
```

★ Whenever you have to change in Original element, Use pointer.

```
}
void Display (struct Array arr)
```

```
{
    int i;
```

```
    printf ("In Elements are \n");
```

```
    for (i = 0; i < arr.length; i++)
```

```
        printf ("%d", arr.A[i]);
```

```
}
```

```
void Append (struct Array *arr, int x)
```

```
{
```

```
    if (arr->length < arr->size)
```

```
        arr->A[arr->length++] = x;
```

```
}
```

call by address
as it going to
modify the
array

arr->A[arr->length] = x;
length++;

```
void insert (struct Array *arr, int index, int x)
```

```
{
    int i;
```

```
    if (index >= 0 && index <= arr->length)
```

```
    {
        for (i = arr->length; i > index; i--)
```

```
            arr->A[i] = arr->A[i-1];
```

arr $\rightarrow$ A[index] = x;

arr $\rightarrow$ length++;

}

}

int main()

{

struct Array arr = {2, 3, 4, 5, 6, 10, 5, 6};

Append(arr, 15); $\rightarrow$ 2, 3, 4, 5, 6, 15

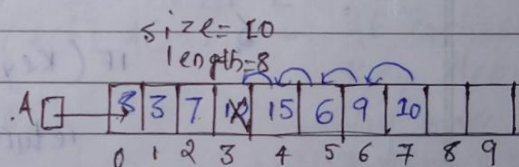
insert(&arr, 5, 10); $\rightarrow$ 2, 3, 4, 5, 6, 10, 15

Display(arr); $\rightarrow$ 2, 3, 4, 5, 6, 10, 15

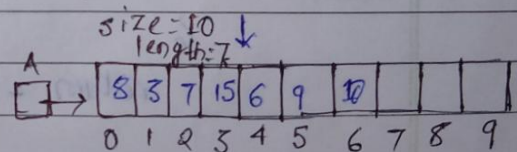
return 0;

}

4. Delete (3)



x = A[index]



For (i = index; i < length - 1; i++) $\rightarrow$ 1

{

A[i] = A[i+1]; $\rightarrow$ 0-7

}

length--;

Best case Min = 2
Worst case Max = n+2

Best case Time $\rightarrow$ O(1)

Worst case Time $\rightarrow$ O(n)

* index shouldn't
beyond length
check it by condition

* if (index > 0 && index < arr->length)

Search

1. Linear Search
2. Binary search

1. Linear Search

Size = 10

8	9	4	7	6	5	10	5	14	2
---	---	---	---	---	---	----	---	----	---

Length = 10

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Successful $\leftarrow$ Key = 5 Best $\rightarrow O(1)$
Unsuccessful $\leftarrow$ Key = 12 Worst $\rightarrow O(n)$

For ($i = 0$; $i < \text{Length}$; $i++$)

{
 IF (Key == A[i])
 return i;

}
return -1; $\rightarrow$ IF invalid index then search unsuccessful

$\rightarrow$ No. of operⁿ in worst possible cases

Average Case $\rightarrow \frac{1+2+3+\dots+n}{n} = \frac{n(n+1)}{2n} = \frac{n+1}{2}$

$\downarrow$
 $O(n)$

★ ★ We generally depend on worst case for analysis.
as For finding average case, we have to do lot of operation

Improving Linear Search

Size = 10

Length = 10

A	8	9	4	7	6	3	10	5	14	2
	0	1	2	3	4	5	6	7	8	9

successful Key = 5

Unsuccessful Key = 12

* IF key element is searched repeatedly then there are methods to reduce no. of step.

1. Transposition

2. Move to Front/Head

1. Transposition For (i = 0; i < Length; i++)

Key = 5

IF (Key == A[i])

swap(A[i], A[i+1]);

return 0;

A	8	9	4	7	6	3	10	5	14	2
	0	1	2	3	4	5	6	7	8	9

→ At each search
Key element
will move one unit
backward

2. Move to Front/Head

For (i = 0; i < Length; i++)

IF (Key == A[i])

swap(A[i], A[0]);

return 0;

A	8	9	4	7	6	3	10	5	14	2
	0	1	2	3	4	5	6	7	8	9

→ At search
Key element
will move at
index 0

Ex:- Linear search

```
#include <stdio.h>
```

```
struct Array
```

```
{ int A[10];
```

```
  int size;
```

```
  int length;
```

```
};
```

```
int LinearSearch (struct Array arr, int key)
```

```
{ int i;
```

```
  for (i = 0; i < arr.length; i++)
```

```
  { if (key == arr.A[i])
```

```
    return i;
```

```
  }
```

```
  return -1; → -ve index means not Found
```

```
}
```

```
int main ()
```

```
{
```

```
  struct Array arr = { {2, 3, 4, 5, 6, 6, 10, 5, 6, 6}
```

```
  printf ("%d", LinearSearch(arr, 4));
```

```
}
```

Ex → Improving Linear Search

#include <stdio.h>

struct Array

{ int A[10];

int size;

int length;

};

void swap (int *x, int *y)

{ int temp;

temp = *x;

*x = *y;

};

int LinearSearch (struct Array *arr, int Key)

{ int i;

for (i=0; i < arr->length; i++)

{ if (Key == arr->A[i])

{

swap (&arr->A[i], &arr->A[i-1]);

return i;

}

}

return -1

int main()

{ struct Array arr = { {2, 3, 4, 5, 6, 10, 56, ...

printf ("Found", LinearSearch (&arr, 5));

}

Page No.

Date: / /

as we have to do
change in array
so, we have to use
pointer.

2. Binary Search

size = 15
key = 18

1	4	8	10	15	18	21	24	27	29	32	34	37	39	41	48
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	l				mid				h						h
					mid				mid						h
					mid				mid						h

Prerequisite

↓
element must be sorted

Key = 8

l	h	mid	mid = $\left\lfloor \frac{l+h}{2} \right\rfloor$
0	14		$\frac{0+14}{2} = 7$
0	6		$\frac{0+6}{2} = 3$
4	6		$\frac{4+6}{2} = 5$
4	4		$\frac{4+4}{2} = 4$ Found

Key = 34

l	h	mid
0	14	7
8	14	11
8	10	9
10	10	10

Key = 25

l	h	mid
0	14	7
0	6	3
4	6	5
6	6	6
7	6	X

↑
If element not present low became greater than high

Algorithm →

```

Algorithm BinSearch(l, h, key)
{
    mid =  $\left\lfloor \frac{(l+h)}{2} \right\rfloor$ ;
    while (l <= h)
    {
        IF (key == A[mid])
            return mid;
        elseif (key < A[mid])
            h = mid - 1;
        else
            l = mid + 1;
    }
    return -1;
}
    
```

Using Recursive Function

Algorithm RBinSearch (l, h, key)

```

{
  IF (l <= h)
  {
    mid =  $\lfloor (l+h)/2 \rfloor$ ;

```

* Discussed earlier IF (key == A[mid])

in case of Tail recursion, it is better to use loop

return mid;

else IF (key < A[mid])

return RBinSearch (l, mid-1, key);

else

return RBinSearch (mid+1, h, key);

return -1;

Tail recursion

Ex:-

```

int BinarySearch (struct Array arr, int key)
{
  int l, mid, h;
  l = 0;
  h = arr.length - 1;
  while (l <= h)
  {
    mid =  $\lfloor (l+h)/2 \rfloor$ ;
    IF (key == arr.A[mid])
      return mid;
    else IF (key < arr.A[mid])
      h = mid - 1;
    else
      l = mid + 1;
  }
  return -1;
}

```

```

int RBinSearch (int a[], int l, int h, int key)
{
  int mid;
  IF (l <= h)
  {
    mid =  $\lfloor \frac{l+h}{2} \rfloor$ ;
    IF (key == a[mid])
      return mid;
    else IF (key < a[mid])
      return RBinSearch (a, mid-1, h, key);
    else
      return RBinSearch (a, mid+1, h, key);
  }
  return -1;
}

```

int mid;

IF (l <= h)

mid = $\lfloor \frac{l+h}{2} \rfloor$;

IF (key == a[mid])

return mid;

else IF (key < a[mid])

return RBinSearch (a, mid-1, h, key);

else

return RBinSearch (a, mid+1, h, key);

return -1;

return -1;

int main()

{ struct Array arr = {2, 3, 4, 5, 6, 10};

BinarySearch (arr, 5);

→ RBinSearch (arr, 0, arr.length, 5);

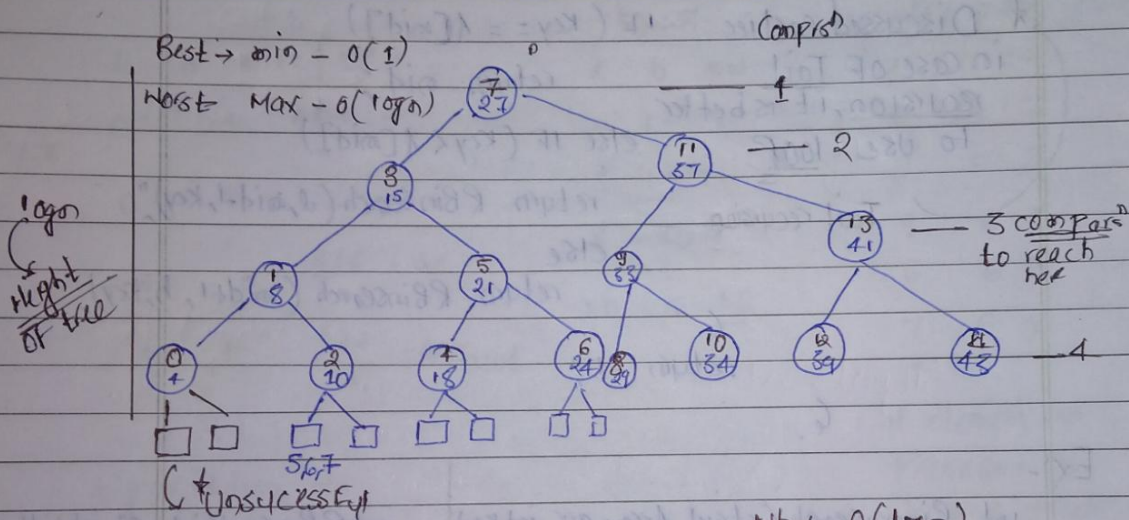
Analysis of Binary Search

1	4	8	10	15	18	21	24	27	29	33	34	37	39	41	43
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Best $\rightarrow \min - O(1)$

Worst Max - $O(\log n)$

Compris



For Unsuccessful attempt

$\log(n+1) \rightarrow \text{Always}$

let's check For $\eta = 16$

$$\log_2 16H = \log_2 17 = 4.2 = 5$$

Maxⁿ Comparison

$$\frac{16}{24} = \frac{2}{3}$$

$$2^4 = 16$$

$$4 = \log_2 16$$

* Actual Formula For order is $\log_2(n+1)$ but we take $\log n$

Why $O(\log n)$

Suppose $\phi = 16$

$$\frac{16}{2} = 8$$

how many times should
be divided to get 1

$$\frac{16}{24} = \frac{2}{3}$$

$$2^4 = 16$$

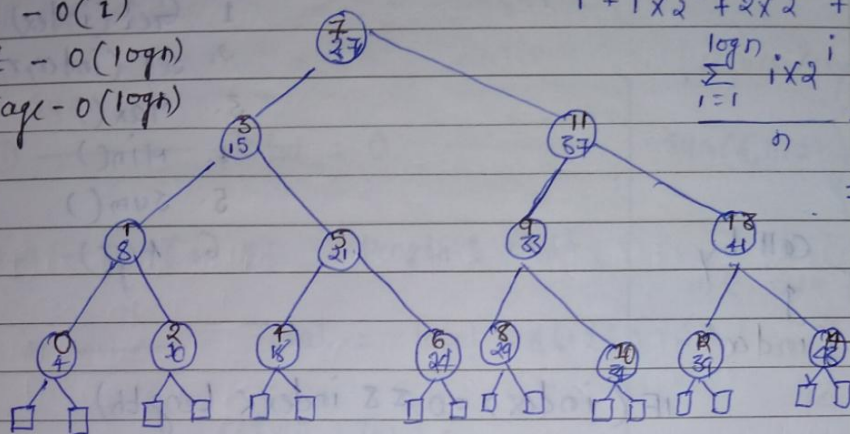
$$4 = \log_2 16$$

Average case analysis

Best - $O(1)$

Worst - $O(\log n)$

Average - $O(\log n)$



no of element in 2 row
 $1 + 1 \times 2 + 2 \times 4 + 3 \times 8$
 $1 + 1 \times 2^1 + 2 \times 2^2 + 3 \times 2^3$

$$\sum_{i=1}^{\log n} i \times 2^i = \log n \times 2^{\log n} = \log n \times n = n \log n$$

$I \rightarrow$ Sum of path of all internal node

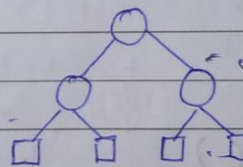
$E \rightarrow$ Sum of path of all external node.

$$E = 1 + I \times 2$$

$e =$ no of external node

$i =$ no. of internal node

$$e = i + 1$$



no of node
 $n = 3$

$$E = 2$$

$$E = 8$$

$$E = i + 2n$$

$$8 = 2 + 2 \times 3$$

$$E = n \log n$$

$$A_s(n) = 1 + \frac{I}{n} = \log n$$

Average Unsuccessful for n

$$A_u(n) = \frac{E}{n+1} = \frac{n \log n}{n+1} = \log n$$

$$\begin{aligned} A_s(n) &= 1 + \frac{I}{n} = 1 + \frac{E - 2n}{n} \quad | \quad E = I + 2n \\ &= 1 + \frac{E}{n} - 2 \\ &= 1 + \frac{n \log n}{n} - 2 \\ &= \log n \end{aligned}$$

size = 15
length = 15

A	4	8	10	15	18	21	24	27	29	33	37	39	41	45	
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

1. Get(index)
2. set(index, x)
3. Max()
4. Min()
5. sum()
6. avg()

call by
↑

1. Get(index)

IF (index $\geq$ 0 & & index $<$ Length)

return A[index];

return -1;

2. set(index, x)

IF (index $\geq$ 0 & & index $<$ Length)

A[index] = x;

3. Max()

1 — max = A[0];

2 — For (i = 1; i $<$ Length; i++)

3 — IF (A[i] > max)

max = A[i];

4

5 — return max;

2n + 1

F(n) = 2n + 1

O(n)

4. Min()

↳ similar to Max()

5. Sum()

① ————— Total = 0

0+1 — For (i = 0; i < Length; i++)

↳

0 ————— Total = Total + A[i];

↳

return Total;

f(n) = 2n + 3

O(n)

recursively

$$\text{Sum}(A, n) = \begin{cases} 0 & n < 0 \\ \text{Sum}(A, n-1) + A[n] & n \geq 0 \end{cases}$$

int Sum(A, n)

↳ IF (n < 0)

return 0;

else

return Sum(A, n-1) + A[n];

call → Sum(A, Length-1);

↓
O(n)

6. Avg()

|
|
|

return Total/n;

→

```
int Get (struct Array arr, int index)
{
    if (index >= 0 && index < arr.length)
        return arr.A[index];
    return -1;
}
```

```
void set (struct Array *arr, int index, int x)
{
    if (index >= 0 && index < arr->length)
        arr->A[index] = x;
}
```

*call by address

```
int Max (struct Array arr)
{
    int max = arr.A[0];
    int i;
    for (i = 1; i < arr.length; i++)
    {
        if (arr.A[i] > max)
            max = arr.A[i];
    }
    return max;
}
```

```
int sum (struct Array arr)
{
    int s = 0;
    int i;
    for (i = 0; i < arr.length; i++)
        s += arr.A[i];
    return s;
}
```

Float avg (struct Array arr)

{ return sum(arr)/arr.length;

}

int main()

{ struct Array arr = {{2, 3, 4, 5, 6, 10, 5, 6;

}

Reverse and Shift an Array

size = 10

length = 10 A [8, 3, 9, 15, 6, 10, 7, 2, 12, 4]

0 1 2 3 4 5 6 7 8 9

1. Reverse

2. Left shift

3. Left Rotate

4. Right shift

5. Right Rotate

1. Reverse

B [4, 12, 2, 7, 10, 6, 15, 9, 3, 8]

0 1 2 3 4 5 6 7 8 9

(a) Method 1 →

Taking Auxillary

array

now Copying in A
for (i = length-1, j = 0; i >= 0; i--, j++)

{

B[j] = A[i];

}

for (i = 0; i < length; i++)

{ A[i] = B[i];

}

}

(b) Method 2 $\rightarrow$ By interchanging.

4	8	3	9	15	6	10	7	2	12	4
	0	1	2	3	4	5	6	7	8	9

$\downarrow$ interchanging or swap

* stops when i become greater than j

For ($i=0, j=\text{length}-1; i < j; i++, j--$)

{

temp = A[i];

A[i] = A[j];

A[j] = temp;

}

2. Left Shift / Rotate

N	E	L	C	O	M	E	-	-	-	-
---	---	---	---	---	---	---	---	---	---	---

N	E	L	C	O	M	E	-	-	-	-
---	---	---	---	---	---	---	---	---	---	---

$\downarrow$
Left shift is used for showing Advertisement in moving left to right (seems like Flow left to right in LCD)

E	L	C	O	M	E	-	-	-	-	N
---	---	---	---	---	---	---	---	---	---	---

$\downarrow$
Left Rotate is used in LED Display. Letters going around & then coming back

Ex →

```
void Reverse (struct Array *arr)
```

```
{ int *B;
```

```
  int i, j;
```

```
  B = (int*) malloc (arr->length * sizeof (int));
```

```
  For (i = arr->length-1, j = 0; i >= 0; i--, j++)
```

```
    B[j] = arr->A[i];
```

```
  For (i = 0; i < arr->length; i++)
```

```
    arr->A[i] = B[i];
```

```
}
```

```
void Reverse2 (struct Array *arr)
```

```
{ int i, j;
```

```
  For (i = 0, j = arr->length-1; i < j; i++, j--)
```

```
    swap (&arr->A[i], &arr->A[j])
```

```
}
```

```
int main()
```

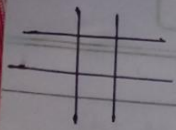
```
{ struct Array arr = {2, 3, 4, 5, 6, 10, 56};
```

```
  Reverse (&arr); / Reverse2 (&arr);
```

```
  Display (arr); → 6, 5, 4, 3, 2
```

```
  return 0;
```

```
}
```



Size = 10
length = 8

Page No.

Date: / /

A	4	8	13	16	20	25	28	33	
	0	1	2	3	4	5	6	7	8

1. Inserting in a sorted Array
2. checking if an Array is sorted
3. Arranging -ve on left side

in Sorted Array

Insert $\rightarrow 18$

A	4	8	18	16	20	25	28	33	
	0	1	2	3	4	5	6	7	8

No need to find position of 18 by comparing from left [By comparison also you have to shift so directly shift them]

$x = 18$

$i = \text{length} - 1$

while ($A[i] > x$)

$\{ A[i+1] = A[i];$

$i--;$

$\{ A[i+1] = x;$

2. isSorted

A	4	18	13	16	20	25	28	33	
	0	1	2	3	4	5	6	7	8

Algorithm isSorted (A, n)

$\{ \text{For } (i = 0; i < n-1; i++)$

$\{ \text{IF } (A[i] > A[i+1])$

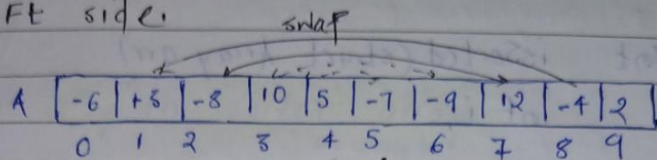
return False;

$\{$

return true;

$\{$

Ex. -ve on left side.



$\leftarrow \text{While}(i < j)$
 $\{$
 $\quad \text{While}(A[i] < 0) \{ i++;$
 $\quad \text{While}(A[j] \geq 0) \{ j--;$
 $\quad \text{If}(i < j)$
 $\quad \quad \{ \text{swap}(A[i], A[j]);$
 $\quad \quad \}$
 $\}$
 $\downarrow$
 $O(n)$

0+2 Comparison
Total

Ex →

```

void InsertSort (struct Array *arr, int x)
{
    int i = arr->length - 1;
    If (arr->length == arr->size)
        return;
    While (i >= 0 & & arr->A[i] > x)
    {
        arr->A[i+1] = arr->A[i];
        i--;
    }
    arr->A[i+1] = x;
    arr->length++;
}
    
```

```

int isSorted (struct Array arr)
{
    int i;
    For (i = 0; i < arr.length - 1; i++)
    {
        IF (arr.A[i] > arr.A[i+1])
            return 0;
        return 1;
    }
}

```

```

void Rearrange (struct Array *arr)
{
    int i, j;
    i = 0;
    j = arr->length - 1;
    While (i < j)
    {
        While (arr->A[i] < 0) i++;
        While (arr->A[j] >= 0) j--;
        IF (i < j) swap (&arr->A[i], &arr->A[j]);
    }
}

```

```

int main()
{
    struct Array arr = {
        // ...
    };
}

```

Merge Array

Condⁿ can be done only in sorted array

Way to Rem

First compare
A & B, which B
is shorter
copy in C

A [3 | 8 | 16 | 20 | 25] m
0 1 2 3 4

[4 | 10 | 12 | 22 | 25] n
0 1 2 3 4

C [3 | 4 | 8 | 10 | 12 | 16 | 20 | 22 | 25 | 25]
0 1 2 3 4 5 6 7 8 9

→ $O(m+n)$

* $m+n$ → denotes merging

→

```
struct Array * Merge(struct Array * arr1, struct Array * arr2)
{
    int i, j, k;
    i = j = k = 0;
```

```
    struct Array * arr3 = (struct Array *) malloc(sizeof(struct Array));
```

```
    while (i < arr1->length && j < arr2->length)
```

```
    {
```

```
        if (arr1->A[i] < arr2->A[j])
```

```
            arr3->A[k++] = arr1->A[i++];
```

```
        else
```

```
            arr3->A[k++] = arr2->A[j++];
```

```
    }
```

```
    i = 0;
```

```
    j = 0;
```

```
    k = 0;
```

```
    while (i < m && j < n)
```

```
    {
        if (A[i] < B[j])
```

```
            C[k++] = A[i++];
```

```
        else
```

```
            C[k++] = B[j++];
```

```
    }
```

```
    for (; i < m; i++)
```

```
        C[k++] = A[i];
```

```
    for (; j < n; j++)
```

```
        C[k++] = B[j];
```

For copying
remaining element
as element remains
left after
applying Merge process

```
For( ; i < arr1->length; i++)
```

```
arr3->A[k++] = arr1->A[i];
```

```
For( ; i < arr2->length; i++)
```

```
arr3->A[k++] = arr2->A[i];
```

```
arr3->length = arr1->length + arr2->length;
```

```
arr3->size = 10;
```

```
return arr3;
```

```
int main()
```

```
struct Array arr1 = {2, 6, 10, 15, 25, 10, 5};
```

```
struct Array arr2 = {5, 4, 7, 18, 20, 10, 5};
```

```
struct Array *arr3;
```

```
Merge(&arr1, &arr2);
```

```
Display(*arr3);
```

```
return 0;
```

H.H.
• Not sorted

Page No.
Date: / /

Set Operations

Elements are not sorted

A:

5	5	10	4	6
---	---	----	---	---

0 1 2 3 4

B:

12	4	7	2	5
----	---	---	---	---

0 1 2 3 4

C:

--	--	--	--	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

Elements are sorted

A:

3	4	5	6	10
---	---	---	---	----

0 1 2 3 4

B:

2	4	5	7	12
---	---	---	---	----

0 1 2 3 4

C:

--	--	--	--	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

1. Union
2. Intersection
3. Difference
4. Set Membership

in all three, for sorted element, we use process of Merge.

only difference with Merge process is that if element are equal copy only one element & increase both indices

1. Union

Not Sorted

Step 1 → First copy all element of A in C

Step 2 → Then for B, for each element search in C. If not there then only copy in

C:

3	5	10	4	6	12	7	2		
---	---	----	---	---	----	---	---	--	--

0 1 2 3 4 5 6 7 8 9
 $n + n \times n \rightarrow$ each element compared
 $\rightarrow n + n \times n \rightarrow n + n^2 \rightarrow O(n^2)$

Step → Compare element of A & B, shorter one copy in C. If both are equal copy only one & move i & j by 1

C:

2	3	4	5	6	7	10	12		
---	---	---	---	---	---	----	----	--	--

0 1 2 3 4 5 6 7 8 9
 $O(m+n) \rightarrow O(n+n) \rightarrow O(n)$
 $\rightarrow O(n)$

2. Intersection

First check
element of A
if present
in B then
Copy inc.

A [3, 5, 10, 4, 6] m
0 1 2 3 4
B [12, 4, 7, 2, 5] n
0 1 2 3 4
C [5, 4, , , , , , , , ,]
0 1 2 3 4 5 6 7 8 9

$n \times m$

$$\rightarrow n \times n \rightarrow O(n^2)$$

Compare i with
j, which
is smaller
move next
only at equality

Condⁿ copy
[copy only
duplicates] K

A [5, 4, 5, 6, 10] m
0 1 2 3 4
B [2, 4, 5, 7, 12] n
0 1 2 3 4
C [4, 5, , , , , , , , ,]
0 1 2 3 4 5 6 7 8 9

$$O(m+n) \rightarrow O(n)$$

3. Difference $\rightarrow A - B$

* First check element
of A in B IF not there
then copy in C

A [3, 10, 6, , , , , , , ,]
0 1 2 3 4 5 6 7 8 9

$$n \times n = n \times n = n^2$$

$$O(n^2)$$

* Compare $A[i]$ with $B[j]$
IF smaller copy it

Otherwise move ahead

* Copy only element of A

A [3, 6, 10, , , , , , , ,]
0 1 2 3 4 5 6 7 8 9

$$O(m+n) \rightarrow O(n)$$

4. Set membership



Same as searching only, check in array IF present
or not.

```

⇒ struct Array * Union(struct Array * arr1, struct Array * arr2)
{
    int i, j, k;
    i = j = k = 0;
    struct Array * arr3 = (struct Array *) malloc(sizeof(struct Array));

```

```

    while (i < arr1->length && j < arr2->length)
    {
        // Difference
        if (arr1->A[i] < arr2->A[j])
        {
            arr3->A[k++] = arr1->A[i++];
        }
        else if (arr2->A[j] < arr1->A[i])
        {
            arr3->A[k++] = arr2->A[j++];
        }
        else
        {
            arr3->A[k++] = arr1->A[i++];
            arr3->A[k++] = arr2->A[j++];
        }
    }

```

```

    for (; i < arr1->length; i++)
        arr3->A[k++] = arr1->A[i];
    for (; j < arr2->length; j++)
        arr3->A[k++] = arr2->A[j];
    arr3->length = k;
    arr3->size = 10;
    return arr3;
}

```

Q 10 25

```

int main()
{
    struct Array arr1 = {2, 6, 10, 15, 25, 10, 56};
    struct Array arr2 = {3, 6, 7, 15, 20, 10, 56};
    struct Array * arr3;
    arr3 = Union(arr1, arr2);
    // 2 3 6 7 10 15 20 25
}

```

⇒ struct Array *intersection(struct Array *arr1, struct Array *arr2)
 {
 int i, j, k;
 i = j = k = 0;

struct Array *arr3 = (struct Array *) malloc(sizeof(struct Array));

while (i < arr1->length && j < arr2->length)

{
 if (arr1->A[i] < arr2->A[j])
 i++;
 else if (arr2->A[j] < arr1->A[i])
 j++;
 else
 {
 arr3->A[k++] = arr1->A[i];
 i++;
 j++;
 }
 }

arr3->length = k;

arr3->size = 10;

return arr3;

int main()

{

struct Array arr1 = {2, 6, 10, 15, 25, 10, 5};

struct Array arr2 = {3, 6, 7, 15, 20, 10, 5};

struct Array *arr3;

arr3 = intersection(&arr1, &arr2); → 6, 15

}

Menu Driver Program For Array

struct array

```
{
    int *A;
    int size;
    int length;
};
```

int main ()

```
{
    struct array arr1;
    int ch;
```

```
    printf("Enter size of Array");
```

```
    scanf("%d", &arr1.size);
```

```
    arr1.A = (int *) malloc(arr1.size * sizeof(int));
```

```
    printf("Menu\n");
```

```
    printf("1. insert\n");
```

```
    printf("2. Delete\n");
```

```
    printf("3. Search\n");
```

```
    printf("4. sum\n");
```

```
    printf("5. Display\n");
```

```
    printf("6. Exit\n");
```

```
    printf("Enter your choice");
```

```
    scanf("%d", &ch);
```

```
    switch(ch)
```

```
{
```

```
    case 1: printf("Enter an element and index");
```

```
            scanf("%d %d", &x, &index);
```

```
            break;
```

Page No. _____
Date: / /

Case 2: printf("Enter index");

scanf("%d", &index);

x = Delete(&arr, index);

printf("Deleted Element is %d\n", x);

break;

Case 3: printf("Enter element to search");

scanf("%d", &x);

index = Linearsearch(&arr, x);

printf("Element index %d\n", index);

break;

Case 4: printf("sum is %d\n", sum(arr));

break;

Case 5: Display(arr);

{

while (ch < 6); → do while loop

return 0;

}

Converting C program to C++

#include <iostream>

Class Array

{ Private:

int *A;

int size;

int length;

Public:

Array()

{

size = 10;

length = 0;

A = new int[size];

}

Array(int sz)

{ size = sz;

length = 0;

A = new int[size];

}

~Array()

{ delete []A;

}

void Display();

void Append(int x);

void insert(int index, int x);

int LinearSearch(int key);

int BinarySearch(int key);

* C Program related to
logic or procedure

while C++ Program is
more related to

designing or organising
the code

int Get (int index);
 void set (int index, int x);
 int Max ();
 int Min ();
 int Sum ();
 float Avg ();
 void Reverse ();
 void Reverse2 ();
 void InsertSort (int x);
 int isSorted ();
 void Rearrange ();

Array * Merge (Array * arr2);
 Array * Union (Array * arr2);
 Array * Intersection (Array * arr2);
 Array * Difference (Array * arr2);

{

void Array::Display ()

{

int i;

printf ("10 Elements are\n");

for (i=0; i < Length; i++)

printf (" %d" A[i]);

}

void Array::Append (int x)

{

if (length < size)

A[length++] = x;

}

void Array::insert (int index, int x)
 {
 int i;
 if (index >= 0 && index <= length)
 {
 for (i = length; i > index; i--)
 A[i] = A[i+1]
 A[index] = x;
 length++;
 }
 }

{

;
 ;
 ;

Array * Merge (Array arr2)
 {
 int i, j, k;
 i = j = k = 0;

Array * arr3 = new Array (length + arr2.length);

while (i < length && j < arr2.length)
 {
 if (A[i] < arr2.A[j])
 arr3->A[k++] = A[i++];
 else
 arr3->A[k++] = arr2.A[j++];

}
 for (; i < length; i++)
 arr3->A[k++] = A[i];
 for (; j < arr2.length; j++)
 arr3->A[k++] = arr2.A[j];

Page No. _____
Date: / /

```
arr3->length = length + arr2.length;
```

```
return arr3;
```

```
{
```

```
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |
```

```
int main()
```

```
{
```

```
    Array *arr1;
```

```
    int ch, sz;
```

```
    int x, index;
```

```
    cout << "Enter size of Array";
```

```
    cin >> sz;
```

```
    arr1 = new Array(szsz);
```

```
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |
```

Note:- For different

data type use

Template class

```
switch (ch)
```

```
case 1: cout << "Enter an element & index"
```

```
cin >> x >> index;
```

```
arr1->insert(index, x);
```

```
break;
```

```
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |  
    |
```

```
while (ch < 0) {
```

```
return 0;
```

Student Exercise

↓

1. Single Missing Element in Sorted Array
2. Multiple " " " " "
3. Missing element in Unsorted Array

1. Single Missing Element in Sorted Array

A	1	2	3	4	5	6	8	9	10	11	12
	0	1	2	3	4	5	6	7	8	9	10

M1 →

$$\frac{n(n+1)}{2} = \frac{12 \times 13}{2} = 78$$

sum = 0;

for (i = 0; i < 11; i++)

{

sum = sum + A[i];

}

$$s = n * (n+1) / 2$$

$$s - \text{sum}$$

$$78 - 71$$

printf ("Missing No. is %d", s - sum);

M2 → A [6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17]
 0 1 2 3 4 5 6 7 8 9 10

$$6 - 0 = 6$$

$$7 - 1 = 6$$

$$8 - 2 = 6$$

$$9 - 3 = 6$$

$$10 - 4 = 6$$

$$11 - 5 = 6$$

$$13 - 6 = 7$$

↑

$$6 + 6 = 12$$

Missing element

$$l = 6$$

$$h = 17$$

$$n = 11$$

$$\text{diff} = l - 0;$$

For (i = 0; i < n; i++)

{

IF (A[i] - i != diff)

{ printf("Missing element is %d", i + diff);
 break;

}

Q. Find Multiple Missing element in an Array

A [6, 7, 8, 9, 11, 13, 15, 16, 17, 18, 19]
 0 1 2 3 4 5 6 7 8 9 10

6 6 6 6 7 7 9 9 9 9 9

$$l = 6$$

$$h = 19$$

$$n = 11$$

$$6 + 4 = 10$$

$$7 + 6 = 13$$

$$8 + 6 = 14$$

diff = 6 - 0;

For (i = 0; i < n; i++)

{ IF (A[i] - i != diff) $O(n)$

{ While (diff < A[i] - i)

{ printf("%d\n", i + diff);

diff++;

}

}

Finding Missing element in Unsorted Array

A [3 7 4 9 12 6 1 11 2 10]
0 1 2 3 4 5 6 7 8 9

$n = 10$

$h = 12$

$n = 10$

STEP 1:-

First of all take an array equal to size to max^m of element & initialise to 0

0 1 2 3 4 5 6 7 8 9 10 11 12

Hashtable / Bitset

takes const time

missing

Time $\rightarrow O(n)$

space $\rightarrow$ it may take equal to bigger element [consumes space]

Now days Machines are providing abundant space in that we can use hashing

```

For (i=0; i<n; i++)
{
    H[A[i]]++;
}
For (i=0; i<=n; i++)
{
    if (H[i] == 0) printf("%d\n", i);
}
O(n)

```

#

1. Finding Duplicates in sorted Array
2. Finding Duplicates in sorted Array

1.- Finding

A	3	6	8	8	10	12	15	15	15	20
	0	1	2	3	4	5	6	7	8	9

n = 10

* i, have to print 15 only one time

last Duplicate = 15

For handling multiple duplicate

lastDuplicate = 0;

For (i = 0 ; i < n-1 ; i++)

{

IF (A[i] == A[i+1] && A[i] != lastDuplicate)

{ PrintF ("%.d\n", A[i]);

lastDuplicate = A[i];

}

Q2. Counting

M1 →

A [3 | 6 | 8 | 8 | 10 | 12 | 15 | 15 | 15 | 20]

0 1 2 3 4 5 6 7 8 9

i j → i j →

↓ ↓
4-2 = 2 9-6 = 3

n = 10

For (i = 0 ; i < n-1 ; i++)

{

IF (A[i] == A[i+1])

{

j = i+1; j = i

While (A[i] == A[j]) j++;

PrintF ("%.d is appearing %d times", A[i], j-i);

i = j-1;

}

}

O(n)

M2 → Finding duplicates Using Hash Table

Scanning A

3	6	8	8	10	15	15	15	20
0	1	2	3	4	5	6	7	8

* Take size = largest num as to get 20

→ in sorted array last element is largest & gives sizes

Taking H const time in incrementing

0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

1) For (i=0; i<n; i++)

H[A[i]]++;

O(n) ←

2) For (i=0; i<max; i++)

IF (H[i] > 1) Print ("rd %d", H[i]);

20)

Finding Duplicates element in Unsorted Array

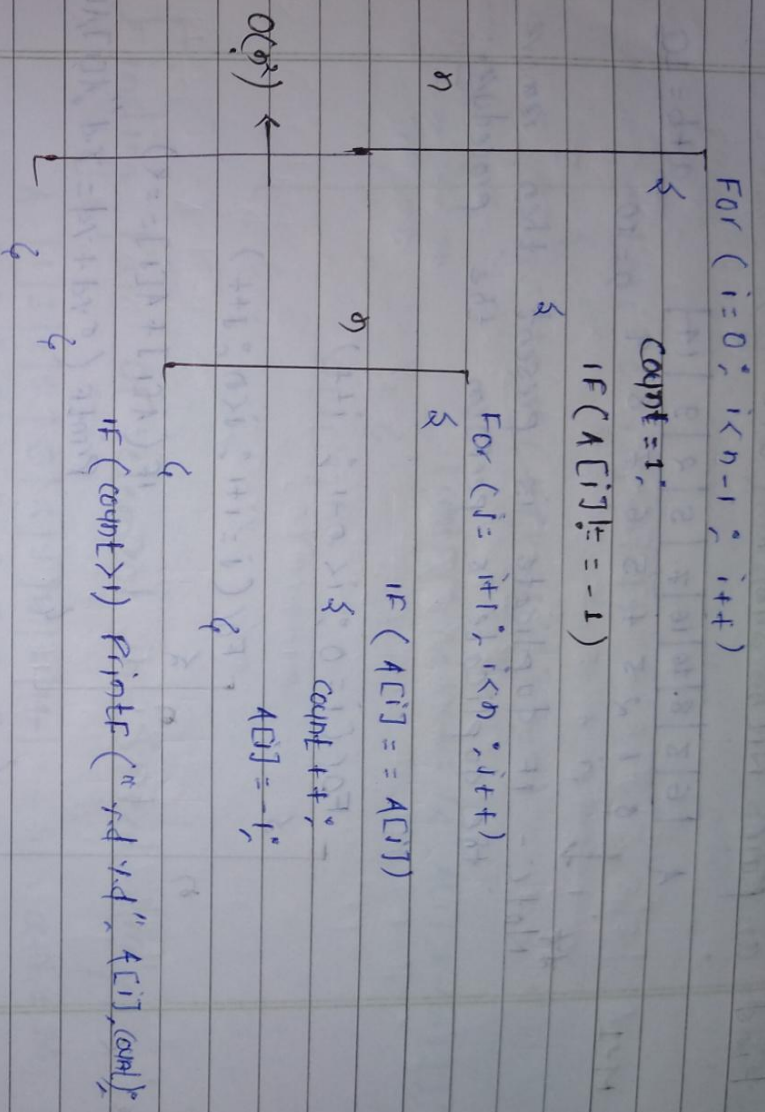
A

8	3	6	4	6	5	6	8	2	7
0	1	2	3	4	5	6	7	8	9

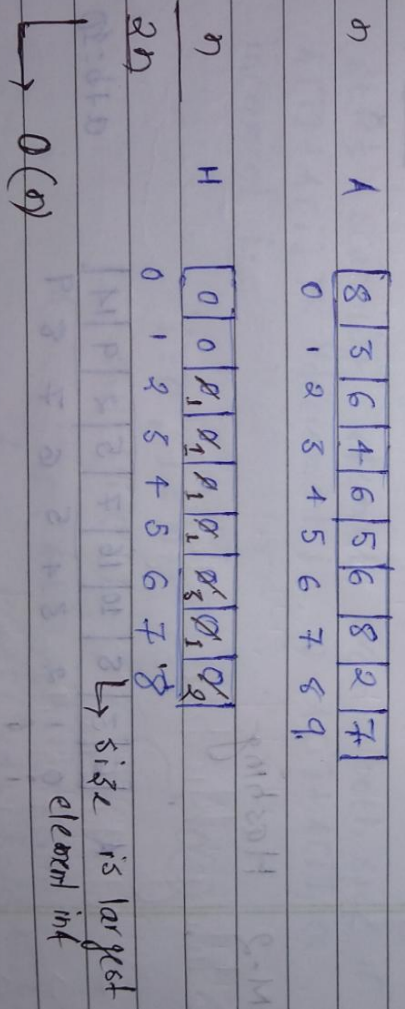
n=9

M1 → Pick each element & search in remaining if present

↓ ** Whenever duplicate is found just mark it as -1 to avoid over counting for next element



M2 Using Hash table



Find a pair with sum k ($a+b=k$)

M-1

A

6	3	8	10	16	7	5	2	9	14
---	---	---	----	----	---	---	---	---	----

 $a+b=10$

↓
i = 0, j = 9

Note: If duplicate is present then remove the duplicate & perform the procedure.

$O(n^2)$ ←

For ($i=0; i < n-1; i++$)
 For ($j=i+1; j < n; j++$)
 IF ($A[i] + A[j] = k$)
 Print ($A[i] + A[j]$)

$n-1 + n-2 + n-3 + \dots + 3 + 2 + 1 = \frac{n(n-1)}{2} \Rightarrow O(n^2)$

M-2 Hashing

A

6	3	8	10	16	7	5	2	9	14
---	---	---	----	----	---	---	---	---	----

 $a+b=10$

i ↓

H

0	0	2	2	1	0	10	1	2	1	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---	---	---

Key: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

Mark 6 to 10 & check

IF 4 is marked or not
 As largest no. is 16
 A is 16.

Note:- Hash table is easier even if duplicate element is present

```

For (i=0; i<n; i++)
{
    if (H[k-A[i]] != 0)
    {
        printf("%d + %d = %d", A[i], k-A[i], k);
        H[A[i]]++;
    }
}

```

Q. Finding Pair of element in Sorted Array

A	[	1	3	4	5	6	8	9	10	12	14	]
i	0	1	2	3	4	5	6	7	8	9		
j												

a+b = 10
n = 10

→ start scanning element from both side if $A[i] + A[j] > k$ decrement j, if $A[i] + A[j] < k$ increment i.

```

i = 0, j = n-1
while (i < j)
{
    if (A[i] + A[j] == k)
        printf("%d + %d = %d", A[i], A[j], k);
        i++;
        j--;
    else if (A[i] + A[j] < k)
        i++;
    else
        j--;
}

```

Finding MAX and MIN in a single scan

A [5 | 8 | 5 | 9 | 6 | 3 | 10 | 7 | -1 | 4]
 0 1 2 3 4 5 6 7 8 9

For Nth min
 comp is satisfied

min = 3, 2
 max = 8, 9, 10

** Non For 3 doesn't
 check with max
 as it is already
 smaller (you will see
 one comp)

min = A[0];
 max = A[0];

Time - $O(n)$

Best IF always
 For (i=1; i<n; i++)
 IF (A[i] < min)
 min = A[i];
 IF (A[i] > max)
 max = A[i];

IF above fails
 1, 2, 3, 5, 8, 9, 10
 Worst

Sliding Window Algorithm

- Introduction
- Identification
- and
- its types

Flow:

- 1) Origin of sliding window
- 2) Brute
- 3) Identity
- 4) Types of sliding window

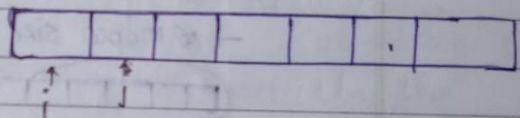
Problem: → arr: 2 3 5 2 9 7 1

			# 3 → size of subarray
2	3	5	→ 10
3	5	2	→ 10
5	2	9	→ 16
2	9	7	→ 18
9	7	1	→ 17

max = ?

Naive

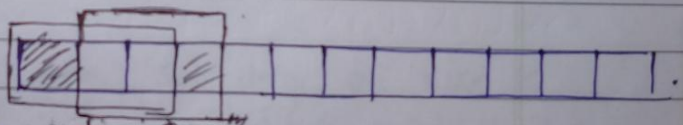
For (i →)
For (j = i ; j < i + 3 ; j++)
Max =



Improvement?

Brute Force

→ ✓
→ ✓
→ ✓
Repetitive work → Optimisation



0 + 1 + 2

1 + 2 + 3 → 1 + 1 + 2 + 3

2 + 3 + 4

Origin

Optimisation

Max^m → Also think of DP

Identification

array string + subarray sub string + largest, Max^m, Min^m

Some times window size is not given but you have to find it
↓
window size??

K = window size

Think atleast one that sliding window will be applicable or not [Not guarantee of sliding window]

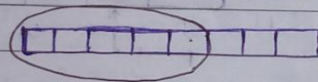
Types

Fixed size window

Variable size window

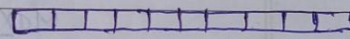
→ window size Fixed

→ window size not given



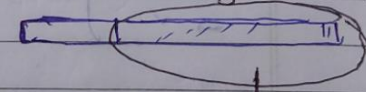
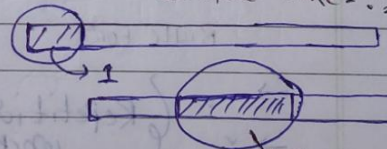
Sum = ??

→ window size = ??



sum given
window size = ??

Used, map, list

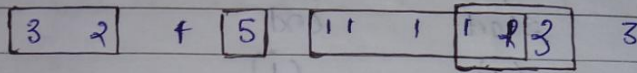


Find largest window size
whose sum is 5

Answer = 8

Ques:- 3, 2, 4, 5, 1, 1, 1, 2, 3, Find largest subarray whose sum is 5

→



$$1 + 1 + 1 = 3$$

Problems

Fixed

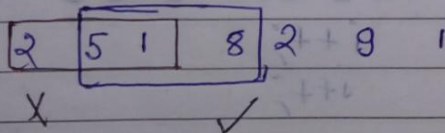
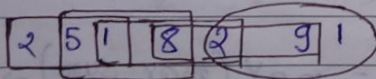
Opposite

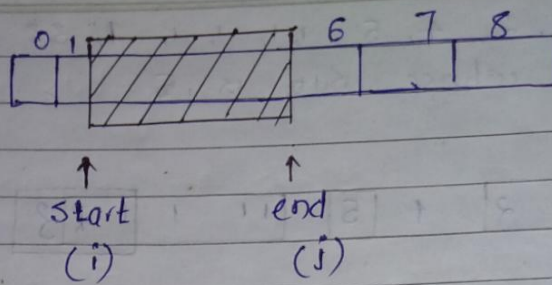
Variable

1. Max/Min subarray of size K
2. 1st -ve in every size of K
3. Count occurrence of anagram
4. Mode of all subarray of size K
5. Max of Min^m for every window size
1. largest/smallest subarray of sum K
2. "largest sub-string of K distinct character"
3. largest of largest sub-string & No repeating character
4. Pick Toy
5. Minimum window subarray

1. size = 7, Max^m sum sub-array of size K
array: 2 5 1 8 2 9 1
window size $K = 3$

→





Window = 4

$$\text{Size of window} = j - i = 5 - 2 = 3 \times$$

$$j - i + 1 \checkmark$$

2	5	1	8	2	9	1
0	1	2	3	4	5	6
i	j					

$$\Rightarrow (j - i + 1) < K \quad \uparrow$$

$$j - i + 1 = K$$

Now start maintaining this window size

while (j < size)

sum = sum + arr[j] ← Calculation

IF (j - i + 1 < K)

j++

else IF (j - i + 1 == K)

{

max = max(max, sum) → Ans

sum = sum - arr[i]

i++

j++

}

6

Q1

2. Given an array and a positive integer K. Find First negative integer for each & every window of size K.

→ ex -

8

5

-8 2 3 -6 10

2

8

12 -1 -7 8 -15 20 16 28

3

→ O/P →

-8 0 -6 -6

-1 -1 -7 -15 -15 0

l → list

→

while (j < size)

{

IF (arr[j] < 0)

{

l.push_back(arr[j]);

}

IF (j - i + 1 < K)

{

j++;

}

ELSE IF (j - i + 1 == K)

{

ans ← calculate

}

}

l

-1 -7

* First in list gives answer,
IF list is empty then
return zero.

calcul

-2 = 3X

✓

window size

ulation

→ Ans
calcul

Identifi

Variable size sliding window

Q1

→ longest subarray of sum K

I/P:-

size: 7

arr: 4 1 1 1 2 3 5

k: 5

longest subarray of sum K = ??

4 1 1 1 2 3 5

4 → 2

2 3 → 2 → 4

5 → 1

ws → max^m (Variable) 1 1 2 → 4

sum → Given

* In Fixed size window, we were checking window size. In variable size window we check given condition.

arr:

4 1 1 1 2 3 5

1 0 1 2 3 4 5 6

i=0 j=0

4 1 1 1 2 3 5

academy

if negative no.s are included

similar Qⁿ → largest subarray with 0 sum

subarray sums equal K
 [1, 1, 1], K = 2 → 2
 [1, 1, 1], 0 → 1
 logic
 S-K → K
 3, 9, -2, 4, -7, 2, 6
 S-K → K

sum = 0

while (j < size)

sum = sum + arr(j);

IF (sum < K)

j++

else IF (sum == K)

max = max(max, j-i+1)

j++

else IF (sum > K)

while (sum > K)

sum = sum - arr(i);
 i++;

j++

return max;

int subarray (sums, k)

int sum = 0

map <int, int> m;

int cnt = 0;

m[0]++ // to take account those subarray that begins with 0

for (int i = 0; i < n; i++)

sum += sums[i];

cnt += m[sum - k];

m[sum]++

return cnt;

-5 8 -14 2 4 19

K = -5

→ 4

if window given

Identif

General Format

calcⁿ

IF (cond < K)

i++

else IF (cond == K)

ans ← calcⁿ

i++

}

else IF (cond > K)

{

while (cond > K)

remove calcⁿ For i

i++

}

i++

return ans

Two pointer Algorithm

$O(n^3)$ changes to $O(n^2)$

$O(n^2)$ change to $O(n)$

$O(n)$ space change to $O(1)$

* sorted array $\begin{cases} \rightarrow \text{Binary search} \\ \rightarrow \text{Two pointer algorithm} \end{cases}$

Q. 2, 4, -1, 3, -4, 0, 3

→ is any triplet exist whose sum is 0?

Naive approach: $O(n^3)$

2, 4, -1, 3, -4, 0, 3

apply -- sorting

-4, -1, 0, 2, 3, 3, 4

*** ~~Two sum~~ Find $x=5$ exist for two number sum. Using two pointer

-4, -1, 0, 2, 3, 3, 4 $\rightarrow O(n)$

0 < 5 $\rightarrow$ i++
3 < 5 $\rightarrow$ i++
4 < 5 $\rightarrow$ i++
6 > 5 $\rightarrow$ j--

write Fⁿ of two sum

For i

Triplet sum

Now solve three sum with help of

two sum

[-1, 0, 2, 3, 3, 4]

search in this sum is any two exist whose value is 4

*** [-4, -1, -1, 0, 1, 2]

To avoid duplicate triplet. $[-1, -1, 2], [-1, 0, 1], [-1, 1, 2]$

For i (To avoid duplicate)

use ()

while (i < j && nums[i] == nums[i+1])
i++;

in two sum

while (i < n-1 && nums[i] == nums[i+1])
i++;

in three sum

Don't forget this

or then

e sum is 0s

Identifi

Q. Merge interval

Input: intervals = [1,3], [2,6], [8,10], [15,18]

Output: [1,6], [8,10], [15,18]

since intervals [1,3] and [2,6] overlaps, merge them into [1,6]

Input: intervals = [1,4], [4,5] Output: [1,5]

Thin
at

Ex:	1,3	2,6	8,10	8,9	9,11	15,18	2,4	16,17
sorted	1,3	2,4	2,6	8,9	8,10	9,11	15,18	16,17

↑
merging with it

temp = [1,3]

Pop (1,6)

temp = [1,4]

[1,6]

[8,9]
[8,10] so on

```

vector<vector<int>> merge(vector<vector<int>> intervals) {
    vector<vector<int>> mergedIntervals;
    if (intervals.size() == 0)
        return mergedIntervals;
    sort(intervals.begin(), intervals.end());
    vector<int> tempInterval = intervals[0];
    for (auto it : intervals) {
        if (it[0] <= tempInterval[1])
            tempInterval[1] = max(it[1], tempInterval[1]);
        else {
            mergedIntervals.push_back(tempInterval);
            tempInterval = it;
        }
    }
    mergedIntervals.push_back(tempInterval);
    return mergedIntervals;
}
    
```

1. Index of an extra element $\rightarrow O(\log n)$

ex:- A[] = {2, 4, 6, 8, 9, 10, 12}

B[] = {2, 4, 6, 8, 10, 12}

O/P: $\rightarrow 4$

A[] = {3, 5, 7, 9, 11, 13}

B[] = {3, 5, 7, 11, 13}

O/P: $\rightarrow 3$

merge them

$\rightarrow$

$l = 0, h = n - 1, ans = n$

$mid = (low + h) / 2$

If (mid of both array is similar) $\rightarrow low = mid + 1$

else $\rightarrow high = mid - 1$ and update the answer to current index, $ans = mid$

Print the index

[1, 5]

Q. Merge without extra space $\rightarrow O((n+m) \log(n+m))$

$n = 4, arr1[] = [1, 3, 5, 7]$

$m = 5, arr2[] = [0, 2, 6, 8, 9]$

Output: $arr1[] = [0, 1, 2, 3]$ $arr2[] = [5, 6, 7, 8, 9]$

$n = 2, arr1[] = [10, 12]$

$m = 3, arr2[] = [5, 18, 20]$

Output: $arr1[] = [5, 10]$ $arr2[] = [12, 18, 20]$

$a1[] = [1, 4, 7, 8, 10]$ $n1 = 5$

$a2[] = [2, 3, 9]$ $n2 = 3$

M1 $\rightarrow$ $\begin{matrix} 2 & 1 \\ \textcircled{1} & 4 & 7 & 8 & 10 \end{matrix}$

$\rightarrow$ $\begin{matrix} & & 2 & & \\ & & \textcircled{2} & & \\ & 3 & & 9 & \end{matrix}$

$\textcircled{4}$ use insertion sort to sorting

Time Complexity $\rightarrow n1 \times n2$

Identifi

M2:- $\rightarrow$ GAP Method $\rightarrow$ Don't have intuition

a1[] = [1, 4, 7, 8, 10] $n1 = 5$

a2[] = [2, 3, 9] $n2 = 3$

gap = $\frac{8}{2}$

[1, 4, 7, 8, 10] [2, 3, 9]

$\downarrow$ swap it

is 1 greater than 10.

we will not swap anything

Next gap $\rightarrow \frac{4}{2} = 2$

[1, 2, 3, 8, 10] 4, 7, 9

Next gap $\frac{2}{2} = 1$

1, 2, 3, 4, 7, 8, 10, 9

[3, 27, 58, 43] $n1 = 4$

[9, 10, 82] $n2 = 3$

always take ceil
Rem $\text{gap} = \lceil \frac{7}{2} \rceil = 4$

Time complexity $\rightarrow \log_2 n \times O(N)$

For gap

For iteration of every gap

space $\rightarrow O(1)$

Q. Major
Given
majority
than
Input:
O/P:-

7 7

on

el

if

cnt =

el =

ituation

Q. Majority element

Given an array of size n . Find the majority element. The majority element is the element that appears more than $\lfloor n/2 \rfloor$ times. If given element always exist

Input: [3, 3, 3]

O/P: 3

Input: [2, 2, 1, 1, 1, 2, 2]

O/P: 2

 $O(n^2)$ $\Downarrow$ $O(n), O(n)$ $\Downarrow$ $O(n), O(1)$ [Moore Voting Algo]

7 7 5 7 5 1 5 7 5 5 7 7 5 5 5 5

IF (cnt == 0)

2

el = arr[i];

6

IF (el == arr[i])

cnt += 1

else

cnt -= 1

IF given not always exist

3 3 4 2 4 4 2 4

cnt = 0 1 2 0 2 2

el = 4

Now again run a loop & check Frequency of that element is greater than $\lfloor n/2 \rfloor$

$$\text{gap} = \frac{8}{2} = 4$$

always take ceiling

$$\lceil \frac{7}{2} \rceil = 4$$

part
→ $O(1)$

O/P

1. Q. Find an element that appears more than $\lfloor n/3 \rfloor$ times

→

1	1	1	3	3	2	2	2
---	---	---	---	---	---	---	---

Boyer * Moore Voting algo

At max we can have two majority element

num1 = -1

num2 = -1

c1 = 0

c2 = 0

For (int el = nums)

if (el == num1) c1++

else if (el == num2) c2++

else if (c1 == 0)

num1 = el

c1 = 1

else if (c2 == 0)

num2 = el

c2 = 1

else

c1--

c2--

num1 = 1, num2 = 2

↓
Now again run loop & check their count

it will also ← IF (num1 != num2)

handle special case that is of -1

IF in array -1 is in place

IF (c1 > n/3) ✓. Push-back (num1)

IF (c2 > n/3) ✓. Push-back (num2)

else

IF (c1 > n/3 || c2 > n/3)

✓. Push-back (num1)

has $L^{2/3}$ times

sorting algorithm

y element

C++

C++

= 0)

C/

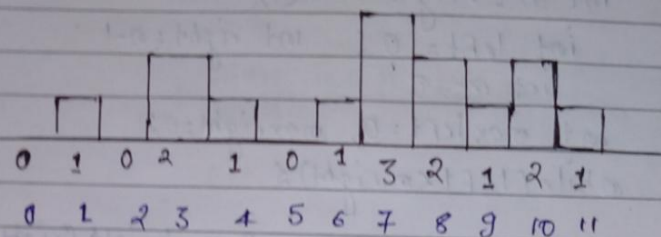
check their

s+)

ns-2)

Pr

Trapping rain water problem



O/P:- 6

Brute Force:- $O(n^2)$

min(left, right) - arr[i], For every(i) look for left bigger & right bigger [using two pointer Technique]

↓

→

Optimize

Pre-Max :- 0 1 1 2 2 2 2 3 3 3 3 3

Suff-Max :- 3 3 3 3 3 3 3 3 2 2 2 1

↓

By doing so I removed nested loop of brute force in which I was traversing left & right.

l=0

 $O(n) + O(n) + O(n)$

r=

Space $\rightarrow O(n)$

↓

Optimize

 $O(n), O(1)$

l=0

res=0

r=n-1

leftmax=0, rightmax=0

if (arr[l] <= arr[r])

{

}

Q.

int trap (vector<int> & height)

int n = height.size();
int left = 0; int right = n-1
int res = 0

int maxleft = 0, maxright = 0;

while (left <= right)

if (height[left] <= height[right])

if (height[left] >= maxleft)

maxleft = height[left];

else

res += maxleft - height[left];

left++;

else

if (height[right] >= maxright)

maxright = height[right];

else

res += maxright - height[right];

right--;

return res;

Two Pointer
approach

Trace it
for logic

it is

handwritten

case

if

is

Q. Remove duplicate From sorted array

arr[] = [1, 1, 2, 2, 2, 3, 3]



Date _____
Page _____

Brute Force

$O(n), O(n)$

↓

using Two pointer

arr = [1, 2, 2, 2, 2, 3, 3]
(i)

inside heap

Heap

stack

right]

right]

right]

right (left)

right]

right]

array

7

